Like this course? Become an expert by

(https://accounts.coursera.org/i/zendesk/courserahelp?return_to=https://learner.coursera.help/hc)

Lessons

All Course Content

(/learn/advanced-data-structures/home/week/2)

Lesson 4 of 4:
Check Your Understanding

Project: Programming Assignment Walk-through 14 min
(/learn/advanced-data-structures/lecture/uND35/project-programming-assignment-walk-through)

Assignment:
Project: Class Design and Graph Search 1h
(/learn/advanced-data-structures/programming/VYrbd/project-class-design-and-graph-search)

Assignment:
Code Design Peer Review 30 min
(/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review)

Review Classmates:
Code Design Peer Review 30 min
(/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review/give-feedback)

Quiz:
End of Week 2 Quiz (complete project and peer review first) 9 questions
(/learn/advanced-data-structures/exam/wpsOd/end-of-week-2-quiz-complete-project-and-peer-review-first)

Previous Lesson
(/learn/advanced-data-structures/lecture/uR44G/in-the-real-world-why-is-code-refactored)

Next Week
(/learn/advanced-data-structures/home/week/3)

Assignment: Code Design Peer Review

You passed!
Congratulations. You earned 15 / 15 points. Review the feedback below and continue the course when you are ready. You can also help more classmates by reviewing their submissions.

Review Classmates' Work (/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review/give-feedback)

Instructions (/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review)

My submission (/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review/submit)

Discussions (/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review/discussions)

Thank you for completing a peer reviewed assignment! Please take this survey to help us improve your experience.
(https://www.surveymonkey.com/r/Z8KFSCC?c=assignmentId%3D2YXl6YyIEeWw2wpCY8gmjw%4010&c=courseId%3DNwQ9txnoEeWDtQoum3sFeQ&c=itemId%3DEHFUs&c=submissionId%3D6JpuZLUaEeWH

README with Commented Code

January 7, 2016

Shareable Link (https://www.coursera.org/learn/advanced-data-structures/peer/EHFUs/code-design-peer-review/review/6JpuZLUaEeWHMgod_NjFNw)

Please enter the description of the classes you created to support your graph structure (i.e. all of the code in the roadgraph package). For each class include a few sentences about the purpose of the class and how it helps store the data required. After your description of your classes, provide a brief justification (1 paragraph, 4-6 sentences) for your overall design. You should use the template provided in this assignment's instructions.

Please see the attached zip for the README file (the description of the classes). Last time I submitted the same files before deadline but somehow the they were not properly attached.

Help Center

Is their description useful in helping you understand and review their code? Note: You must finish your review of their code before you can really answer this question. We recommend you read their description, then answer the prompts below, then come back to respond to this prompt.

1 pt

Yes

0 pts

No

Help your peer! Complete the following prompt: "The description would have been more useful to me if..."

Vamsi
Well commented code, good job!

jacobo bouzada jaureguizar
Good idea to copy the generated javadoc !
Great work !
Congrats !

Антон Гетьманчук
Awesome description mate!

Please upload a zip file containing all of the code in your mapgraph package.

README_with_Commented_Code (https://s3.amazonaws.com/coursera-uploads/peer-review/NwQ9txnoEeWDtQoum3sFeQ/9398e79a6e9812b646d2ed2cde496007/ReadmeWithCode.zip)

README with Commented Code

Do the objects make sense and do the data and methods in those objects go together?

- 2 pts
  Yes
- 1 pt
  Sort of, but there are some aspects don't seem to fit
- 0 pts
  No, not really

Have they developed an appropriate number of classes?

- 1 pt
  Yes
- 0 pts
  They did not develop enough. They should have created more.
- 0 pts
  They developed too many. They should have created fewer.

Are their classes used appropriately in the MapGraph class to store the necessary data?

- 1 pt
  Yes
- 0 pts
  No

Do they use appropriate data structures throughout their code to store their data for efficient retrieval (e.g. is it fast to access vertices and edges when performing the search)?

- 2 pts
  Yes
- 1 pt
  Sometimes. But some data structures could be improved.
- 0 pts
  No

Are class interfaces clean (appropriate access and methods, private data (or data structures) are not exposed)?

- 2 pts
  Yes.
- 1 pt
  Sometimes, but they could be improved.
- 0 pts
  No (or almost never)

Are methods are short and structured appropriately? E.g. do they use helper methods where appropriate?

- 2 pts
  Yes
- 0 pts
  Sometimes
- 0 pts
  No (or almost never)

Is their code easy to read? Consider the following when answering this question: Is there no redundant code? Do they use meaningful variable names and appropriate indentation and spacing?

- (selected) 2 pts
  Yes
- 1 pt
  It's OK, but it could be better
- 0 pts
  No, it's very hard to read

Do they include appropriate comments? That is: Does each method have a header comment that clearly describes what the method does, including its input and output? Are in-line comments used where needed? (note: good code should not need many in-line comments).

- (selected) 2 pts
  Yes, always (or almost always)
- 1 pt
  Sometimes, but it could be better
- 0 pts
  No (or almost never)

Help your peer! Explain your reasoning for any of your responses above. Remember to keep your comments constructive and respectful.

Vamsi

few comments.
1. you don't really need to maintain these counts: numVertices, numEdges. These can be derived from the lists.

2. it is not entirely clear why you need this:
private Map<GeographicPoint, Map<GeographicPoint, Road>> roadMap;
It is being used in addEdge but then you dont really make use of this data structure for anything in your code and it increases space complexity of your code.

3. in your BFS algorithm, I would move this check:

```
if (point.equals(goal)) {       // goal found
                    break;
                }
```

before the for loop that checks the neighbors. Reason being, if the node you just dequeued is the goal node then you can return the path and dont waste time checking all the neighbors as it is not necessary. in a large graph with lots of neighbors this will add to the execution time.

Other than these comments, excellent job at the implement!

jacobo bouzada jaureguizar

You made a different approach to resolve the assignment with just one extra class : Road.
Your code is clean and the comments are explicit.
You have made an excellent work.
Congratulations!

Антон Гетьманчук

Elegant data structure and a lot of comments.

Edit submission

Comments
Visible to classmates

share your thoughts...